

Guía de trabajo: analizador léxico con Flex

Cuaderno de diseño y programación socrática

Compiladores, semestre 2027-1

Esta guía transforma el enunciado en una ruta de trabajo. No es un documento listo para entregar ni contiene las expresiones regulares o el código resueltos. Su propósito es ayudarte a tomar cada decisión, escribirla con tus palabras y comprobarla antes de avanzar.

Qué debe existir al final

El producto tiene dos partes: un único programa fuente definitivo de Flex con el código C necesario, y un documento de entrega con descripción del problema, expresiones regulares, planificación, diseño, pruebas, instrucciones y conclusiones.

Convierte el enunciado en una lista verificable:

- Reconocer las nueve clases, numeradas de 0 a 8.
- Conservar los valores exactos de cada catálogo.
- Registrar identificadores sin duplicarlos.
- Registrar cada aparición de una cadena o real, incluso si se repite.
- Convertir una constante entera a su valor numérico.
- Ignorar comentarios de línea y de bloque.
- Informar errores con número de línea y continuar.
- Aceptar por línea de comandos el archivo fuente.
- Mostrar las tres tablas y la secuencia de tokens al terminar.

Pregunta de control: ¿cada requisito puede señalarse en una función, regla o prueba concreta?

Decisiones que conviene aclarar primero

Anota tu interpretación y, si es posible, confírmala con la profesora antes de fijar las expresiones regulares.

- El primer ejemplo de identificador parece tener un espacio antes de `_I`. ¿Es un error tipográfico? ¿Sería compatible con la descripción formal?
- Los enteros usan `n` y `p`, pero los reales de ejemplo usan `-` y no muestran `+`. ¿Qué signos exactos aceptarás para cada clase?
- ¿La notación científica exige parte decimal o también permite `32e-3`, como indica el ejemplo?
- Dentro de `<...>`, ¿un `>` sin mecanismo de escape siempre termina la cadena?
- ¿Qué significa que `@n`, `@f`, `@d` y `@t` pueden referenciar valores: secuencias internas, componentes separados o sustituciones?
- ¿Qué debe informarse ante un comentario o una cadena sin cierre?
- Si un prefijo parece iniciar un token válido y después falla, ¿qué fragmento se reportará como un solo error?

Construcción de expresiones regulares

Trabaja una clase a la vez. Para cada clase llena cuatro columnas: lenguaje en palabras, bloques pequeños, expresión completa y fronteras o casos inválidos.

Clase 0: palabras reservadas

Hay 17 palabras y sus valores 0 a 16 son inamovibles. ¿Conviene una regla por palabra o una regla conjunta con búsqueda? ¿Cómo garantizas que el valor coincide con la posición dada? ¿Se distinguen mayúsculas y minúsculas?

Clase 1: identificadores

Descompón la forma en prefijo, cuerpo y sufijo. ¿Puede el cuerpo estar vacío? ¿El guion bajo pertenece también al cuerpo? ¿Cómo impones 32 caracteres totales? ¿Cómo reportas completa una forma de 33 caracteres? Calcula primero el máximo del cuerpo.

Clases 2 a 5: operadores y símbolos

Copia el orden exacto de los catálogos y marca todo lexema que sea prefijo de otro. ¿Qué elige Flex ante dos coincidencias que empiezan juntas? Si tienen igual longitud, ¿qué papel juega el orden? ¿Cómo pruebas los lexemas solapados? ¿Una función común puede obtener el índice de cualquier catálogo?

Clase 6: constantes enteras

Separa cero, positivo distinto de cero y negativo. ¿Cómo impides formas con signo de cero? ¿Aceptarás ceros iniciales? ¿Cómo conviertes y detectas desbordamiento? ¿Qué partes del lexema deben excluirse o interpretarse durante la conversión?

Clase 7: constantes reales

Construye por separado forma decimal, exponente y signo. ¿Cuántos dígitos exige cada lado del punto? ¿Qué partes son opcionales en 32e-3? ¿Aceptas e y E, y cuáles signos? Si el dato se almacena como cadena, ¿es necesaria una conversión numérica?

Clase 8: constantes cadena

¿Cómo representas ASCII 32 a 126 sin consumir el delimitador final? ¿Es válida <>? ¿Se permiten saltos de línea? ¿Las secuencias con @ requieren trato especial? Decide si la tabla guarda delimitadores o sólo contenido y explica por qué.

Espacios, comentarios y errores

Diseña el descarte de uno o más espacios, tabuladores o saltos. Compara para comentarios de bloque una sola expresión con un estado exclusivo de Flex: ¿cuál detecta mejor el fin de archivo sin cierre y mantiene la línea? La estrategia final de error debe consumir siempre algo. ¿Por qué eso garantiza progreso?

Diseño del programa en C

Token

Debe guardar clase y valor. ¿Qué tipos corresponden? ¿Cómo crecerá una secuencia de longitud desconocida? ¿Qué función agregará un token sin repetir lógica en nueve acciones?

Tabla de símbolos

Cada entrada contiene posición, nombre y tipo inicialmente -1. ¿La posición se almacena o se deduce del índice? ¿Copiarás el nombre o conservarás yytext? Investiga qué ocurre con yytext tras la siguiente coincidencia. Justifica búsqueda lineal u otra técnica. Diseña una sola operación buscar-o-insertar que siempre devuelva la posición.

Tablas de literales

Cada entrada contiene posición y dato como cadena; hay una tabla para reales y otra para cadenas. ¿Por qué insertar no debe buscar duplicados? ¿Pueden compartir tipo y función? ¿Quién reserva, copia y libera cada dato?

Catálogos fijos

Una representación debe preservar el orden dado y convertir lexema en índice. ¿Usarás arreglos de cadenas? ¿Una búsqueda sirve a los cinco catálogos? ¿Cómo evitas escribir una longitud que pueda desactualizarse?

Memoria dinámica

Para cada arreglo define tamaño usado y capacidad. ¿Cuándo debe crecer? ¿Qué política evita reservar en cada inserción? ¿Cómo evita un apuntador temporal perder el bloque original al fallar una redimensión? Enumera todo lo que debe liberarse al final y en salidas por error.

Flujo completo

1. Validar la cantidad de argumentos.
2. Abrir el archivo indicado y comprobar el resultado.
3. Asignar el flujo de entrada de Flex.
4. Inicializar estructuras.
5. Analizar hasta fin de archivo y acumular resultados.
6. Cerrar el archivo.
7. Mostrar símbolos, literales y tokens.
8. Liberar toda la memoria propia.
9. Devolver un estado que distinga éxito y distintos fallos.

Si ocurre un error léxico recuperable, ¿debe interrumpirse alguno de esos pasos?

Orden de implementación recomendado

1. Crear el esqueleto y comprobar entrada por línea de comandos.
2. Activar y comprobar conteo de líneas.
3. Ignorar delimitadores y reconocer errores carácter por carácter.
4. Definir tipos y operaciones de almacenamiento.
5. Implementar catálogos fijos y búsquedas.
6. Añadir clases 0 y 2 a 5, verificando valores exactos.
7. Añadir identificadores con buscar-o-insertar.
8. Añadir enteros y comprobar conversión y cero.
9. Añadir reales y cadenas con inserción incondicional.
10. Añadir comentarios, incluido fin de archivo inesperado.
11. Imprimir resultados y liberar memoria.
12. Ejecutar integración y revisar memoria con herramientas apropiadas.

Matriz mínima de pruebas

Escribe antes el resultado esperado: tokens, posiciones, errores y línea.

Familia	Casos que debes preparar
Reservadas	Las 17; mayúscula; varias consecutivas
Identificador	Cuerpo mínimo; longitud 32 y 33; repetido; carácter prohibido
Asignación	Los 6; consecutivos; prefijos incompletos
Relacionales	Los 6; prefijos parecidos; secuencia inválida
Aritméticos	Los 7; lexemas solapados
Especiales	Los 7; combinaciones sin espacios
Enteros	Cero; positivo; negativo; cero con signo; sin i; enorme
Reales	Ejemplos; exponentes; signos; punto solo; exponente incompleto
Cadenas	Vacía; espacios; @; límite ASCII; sin cierre; salto
Comentarios	Línea; bloque; varias líneas; marcadores; sin cierre
Integración	Sin espacios; errores entre tokens; archivo vacío

Comprueba además: un identificador repetido mantiene una posición y una entrada; un literal real o cadena repetido crea dos; después de un error se reconoce el token válido siguiente; las líneas siguen correctas tras comentarios y rechazos.

Documento de entrega

1. Descripción del problema con tus palabras, restricciones, expresiones regulares y casos frontera.
2. Propuesta y fases: actividades, responsables y calendario; estructuras, memoria, búsqueda e inserción; casos y resultados de prueba.
3. Ejecución: requisitos, generación con Flex, compilación, invocación y salida.
4. Conclusión por participante: decisiones, dificultades, evidencia de aprendizaje y mejoras.

La fecha indicada es el 29 de septiembre de 2026. Reserva tiempo para probar el paquete final desde una carpeta limpia.

Forma de trabajar con el archivo guía

Usa `analizador.l` como cuaderno. Contesta sus comentarios, escribe la parte mínima correspondiente, compila, ejecuta casos pequeños y conserva las pruebas que descubran errores. Sólo entonces avanza. Cuando pidas ayuda, muestra el bloque, el comando, la entrada mínima y el resultado: así podremos razonar sobre una decisión concreta sin sustituir tu trabajo.